COMP 4531 – Neural Networks & Deep Learning

Final Project Report

Title: Automated Classification of Steel Surface Defects using CNNs and Transfer Learning

Author: Michael Ghattas

Quarter: Q4–2025

Abstract

Manual visual inspection of steel surface quality is slow, subjective, and prone to error. This project investigates whether deep learning can automatically classify surface defects from images using convolutional neural networks (CNNs). The NEU surface defect dataset, which contains six defect categories, is used. The project follows a progression from simple baselines (logistic regression, MLP) to custom CNNs and finally transfer learning with EfficientNetB0. Baselines perform poorly, CNNs perform better, and transfer learning achieves the best test accuracy and F1 scores, showing CNNs' strength for industrial inspection tasks.

1. Introduction & Motivation

Modern steel manufacturing relies on high-quality surfaces. Defects such as scratches, scale, and pitting can weaken products and cause rework. Manual inspection is inconsistent and unscalable. CNNs can automate and improve reliability. The project goal is to build and evaluate models that classify steel defects using NEU-DET.

2. Problem Definition

The task is a supervised multiclass classification problem with six labels: crazing, inclusion, patches, pitted_surface, rolled-in_scale, and scratches. Input: 200×200 RGB image; Output: one of six defect classes.

3. Why Deep Learning is Appropriate

Steel defect textures are local and subtle. CNNs exploit spatial locality and hierarchical features. Pretrained models such as EfficientNetB0 leverage learned features from ImageNet, improving generalization with limited data.

4. Dataset & Exploration

NEU-DET dataset contains 1,800 grayscale images (300 per class). Split: 64% train, 16% validation, 20% test. Visualization confirmed class balance and label correctness.

5. Preprocessing & Data Pipeline

Images loaded via TensorFlow’s image_dataset_from_directory, normalized to [0,1], and augmented with flips, rotations, zooms, and contrast adjustments.

6. Models

Baselines: Logistic Regression (0.36 test accuracy) and MLP (0.41). CNN v1 (3 conv blocks) achieved 0.88 test accuracy. CNN v2 (regularized) underperformed (~0.17). Transfer learning with EfficientNetB0 reached 0.83 accuracy with strong per-class metrics.

7. Training & Optimization

All CNNs trained with Adam, batch size 32, early stopping, and learning-rate scheduling. CNN v1 and EfficientNet converged smoothly; CNN v2 showed underfitting due to heavy regularization.

8. Results Summary

| Model | Test Accuracy | | Logistic Regression | 0.36 | | MLP | 0.41 | | CNN v1 | 0.88 | | CNN v2 | 0.17 | | EfficientNetB0 | 0.83 |

9. Error Analysis & Discussion

Hardest class: pitted_surface (confused with rolled-in_scale). CNN v1 generalized best; EfficientNetB0 showed strong robustness without fine-tuning. Overfitting controlled via regularization and data augmentation.

10. Limitations

Dataset small (1,800 images). Transfer model not fine-tuned. Patch-level classification lacks full context. Larger context or segmentation models may improve results.

11. Conclusions

CNNs, particularly EfficientNet-based architectures, can accurately classify steel surface defects. CNN v1 achieved 0.88 test accuracy; EfficientNetB0 achieved 0.83. These validate CNNs’ industrial applicability for automated quality inspection.

12. Future Work

Fine-tuning pretrained layers, using focal loss, collecting larger datasets, integrating into real-time systems, and adding explainability (Grad-CAM).

13. References & Acknowledgements

- NEU-DET dataset. - TensorFlow/Keras CNN & EfficientNet docs. - Scikit-learn baselines. All analysis and implementation performed independently by Michael Ghattas for COMP 4531.

COMP 4531 - Final Project

Automated Classification of Steel Surface Defects Using CNNs

Author: Michael Ghattas
Quarter: Q4 2025

1. Problem Statement and Project Goals

In high-volume manufacturing (e.g., aerospace and steel production), surface quality inspection is critical for safety and cost. Manual visual inspection is inconsistent, slow, and error-prone. This project uses **deep learning** to automatically classify steel surface defects from images.

Goal:
Given a grayscale image of a steel surface, predict one of six defect classes from the NEU-DET dataset:

- `crazing`
- `inclusion`
- `patches`
- `pitted_surface`
- `rolled-in_scale`
- `scratches`

We will:

1. Explore and clean the data and justify why deep learning is appropriate.
2. Build **baseline models** (logistic regression, MLP) on flattened pixels.
3. Implement a **custom CNN** from scratch and analyze under/overfitting.
4. Refine the model using regularization and data augmentation.
5. Implement **transfer learning** (EfficientNetB0) and compare performance.
6. Analyze errors and discuss limitations and future work.

In [1]:

```python
import os
import warnings
from typing import Tuple

import numpy as np
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models, regularizers

from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.utils import class_weight

plt.rcParams["figure.figsize"] = (8, 6)
plt.rcParams["axes.grid"] = True

# Silence the urllib3 NotOpenSSLWarning if present
try:
    from urllib3.exceptions import NotOpenSSLWarning
    warnings.filterwarnings("ignore", category=NotOpenSSLWarning)
except Exception:
    pass

print("TensorFlow version:", tf.__version__)
print("Num GPUs Available:", len(tf.config.list_physical_devices('GPU')))
```

```
/Users/magmac/Downloads/.venv/lib/python3.9/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' module is
compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
  warnings.warn(
TensorFlow version: 2.16.2
Num GPUs Available: 1
```

In [ ]:

```python
# Paths and constants
DATA_DIR_TRAIN_BASE = "/Users/magmac/Downloads/NEU-DET/train/images"
DATA_DIR_TEST        = "/Users/magmac/Downloads/NEU-DET/validation/images"

IMG_HEIGHT = 200
IMG_WIDTH  = 200
BATCH_SIZE = 32
SEED = 42
NUM_CLASSES = 6

print("Train base exists:", os.path.isdir(DATA_DIR_TRAIN_BASE))
print("Test base exists: ", os.path.isdir(DATA_DIR_TEST))
```

```
Train base exists: True
Test base exists:  True
```

In [ ]:

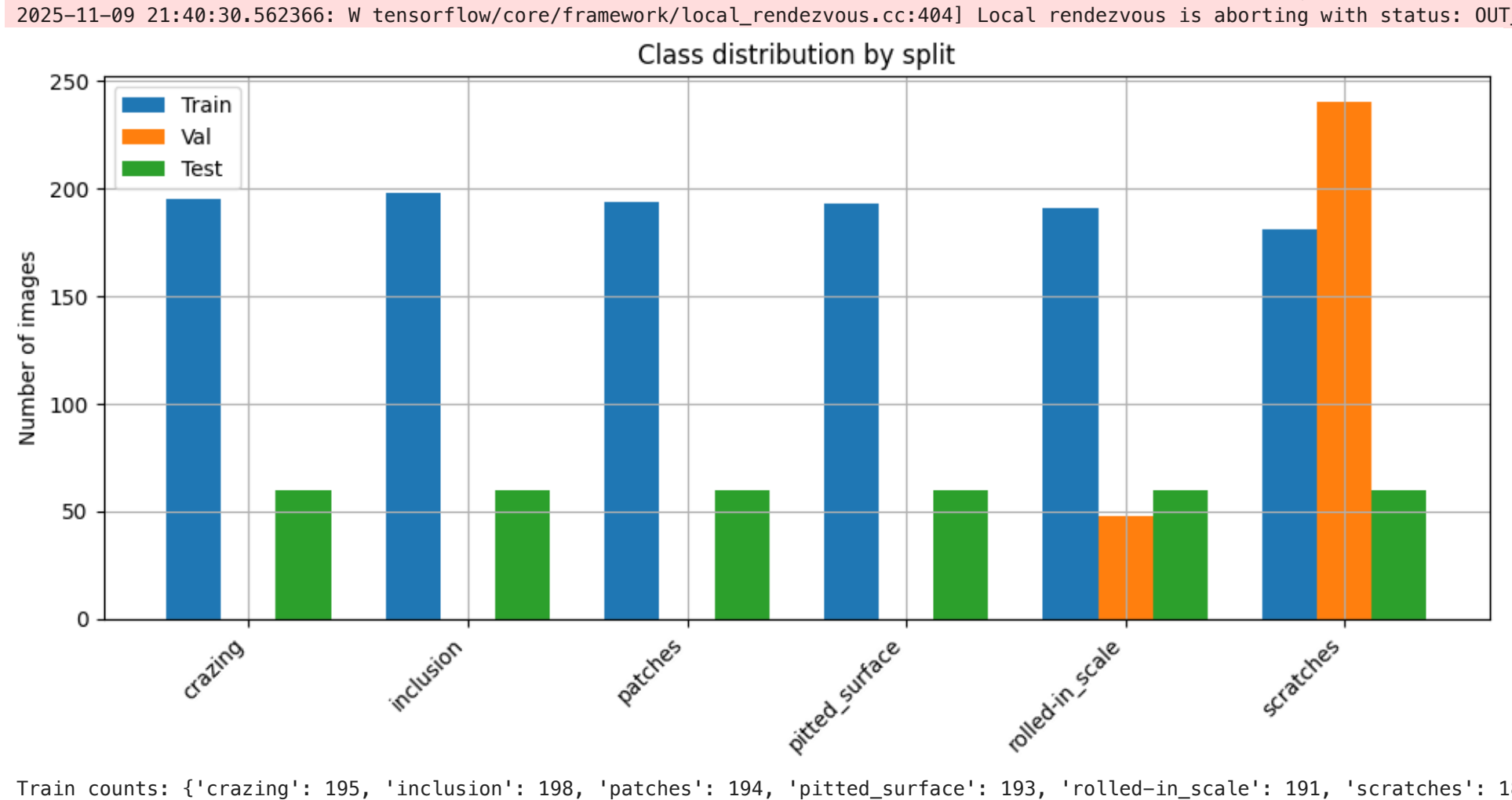
```python
# Create train / validation / test datasets
train_ds_raw = keras.utils.image_dataset_from_directory(
    DATA_DIR_TRAIN_BASE,
    labels="inferred",
    label_mode="categorical",          # one-hot labels
    validation_split=0.2,              # 80% train, 20% val
    subset="training",
    seed=SEED,
    image_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    shuffle=True,
)

val_ds_raw = keras.utils.image_dataset_from_directory(
    DATA_DIR_TRAIN_BASE,
    labels="inferred",
    label_mode="categorical",
    validation_split=0.2,
    subset="validation",
    seed=SEED,
    image_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    shuffle=False,
)

test_ds_raw = keras.utils.image_dataset_from_directory(
    DATA_DIR_TEST,
    labels="inferred",
    label_mode="categorical",
    image_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    shuffle=False,
)

class_names = train_ds_raw.class_names
print("Class names:", class_names)

# Add prefetching for performance
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds_raw.prefetch(AUTOTUNE)
val_ds   = val_ds_raw.prefetch(AUTOTUNE)
test_ds  = test_ds_raw.prefetch(AUTOTUNE)

print("Train batches:", tf.data.experimental.cardinality(train_ds).numpy())
print("Val batches:  ", tf.data.experimental.cardinality(val_ds).numpy())
print("Test batches: ", tf.data.experimental.cardinality(test_ds).numpy())
```

```
Found 1440 files belonging to 6 classes.
Using 1152 files for training.
2025-11-09 21:40:29.457086: I metal_plugin/src/device/metal_device.cc:1154] Metal device set to: Apple M3
2025-11-09 21:40:29.457123: I metal_plugin/src/device/metal_device.cc:296] systemMemory: 24.00 GB
2025-11-09 21:40:29.457128: I metal_plugin/src/device/metal_device.cc:313] maxCacheSize: 8.00 GB
2025-11-09 21:40:29.457150: I tensorflow/core/common_runtime/pluggable_device/pluggable_device_factory.cc:305] Could not identify NUMA node of platform GPU ID 0, defaulting
to 0. Your kernel may not have been built with NUMA support.
2025-11-09 21:40:29.457164: I tensorflow/core/common_runtime/pluggable_device/pluggable_device_factory.cc:271] Created TensorFlow device (/job:localhost/replica:0/task:0/de
vice:GPU:0 with 0 MB memory) -> physical PluggableDevice (device: 0, name: METAL, pci bus id: <undefined>)
Found 1440 files belonging to 6 classes.
Using 288 files for validation.
Found 360 files belonging to 6 classes.
Class names: ['crazing', 'inclusion', 'patches', 'pitted_surface', 'rolled-in_scale', 'scratches']
Train batches: 36
Val batches:   9
Test batches:  12
```

In [ ]:

```python
# Exploratory data analysis – class distribution
def count_labels(ds) -> np.ndarray:
    counts = np.zeros(len(class_names), dtype=int)
    for _, labels in ds:
        idx = np.argmax(labels.numpy(), axis=1)
        for i in idx:
            counts[i] += 1
    return counts

train_counts = count_labels(train_ds_raw)
val_counts   = count_labels(val_ds_raw)
test_counts  = count_labels(test_ds_raw)

x = np.arange(len(class_names))
width = 0.25

plt.figure(figsize=(10, 5))
plt.bar(x - width, train_counts, width, label="Train")
plt.bar(x,         val_counts,   width, label="Val")
plt.bar(x + width, test_counts,  width, label="Test")
plt.xticks(x, class_names, rotation=45, ha="right")
plt.ylabel("Number of images")
plt.title("Class distribution by split")
plt.legend()
plt.tight_layout()
plt.show()

print("Train counts:", dict(zip(class_names, train_counts)))
print("Val counts:  ", dict(zip(class_names, val_counts)))
print("Test counts: ", dict(zip(class_names, test_counts)))
```

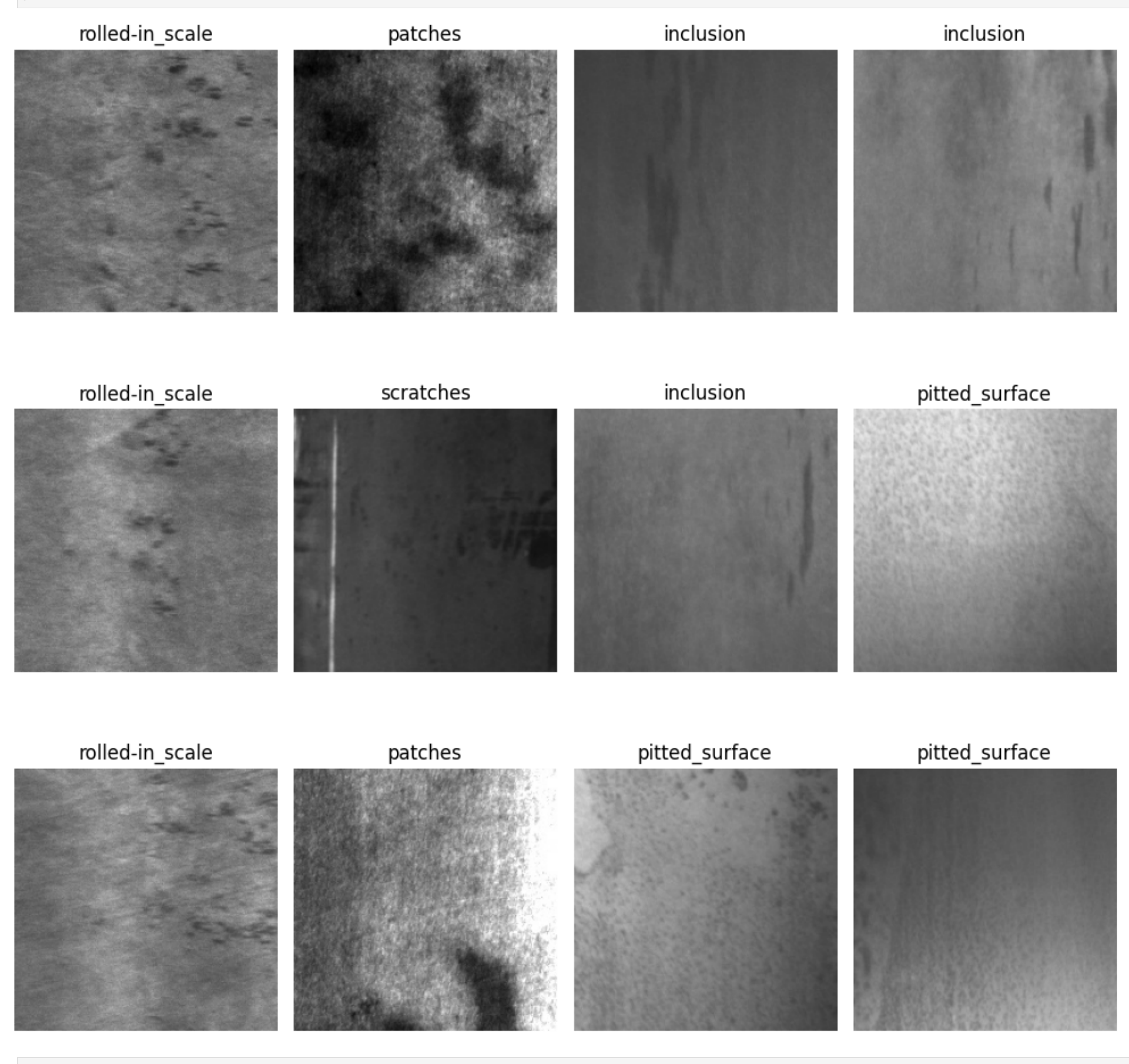
```
2025-11-09 21:40:30.299379: W tensorflow/core/framework/local_rendezvous.cc:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
2025-11-09 21:40:30.479527: W tensorflow/core/framework/local_rendezvous.cc:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
2025-11-09 21:40:30.562366: W tensorflow/core/framework/local_rendezvous.cc:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
```


Class distribution by split

```
Train counts: {'crazing': 195, 'inclusion': 198, 'patches': 194, 'pitted_surface': 193, 'rolled-in_scale': 191, 'scratches': 181}
Val counts:   {'crazing': 0, 'inclusion': 0, 'patches': 0, 'pitted_surface': 0, 'rolled-in_scale': 48, 'scratches': 240}
Test counts:  {'crazing': 60, 'inclusion': 60, 'patches': 60, 'pitted_surface': 60, 'rolled-in_scale': 60, 'scratches': 60}
```

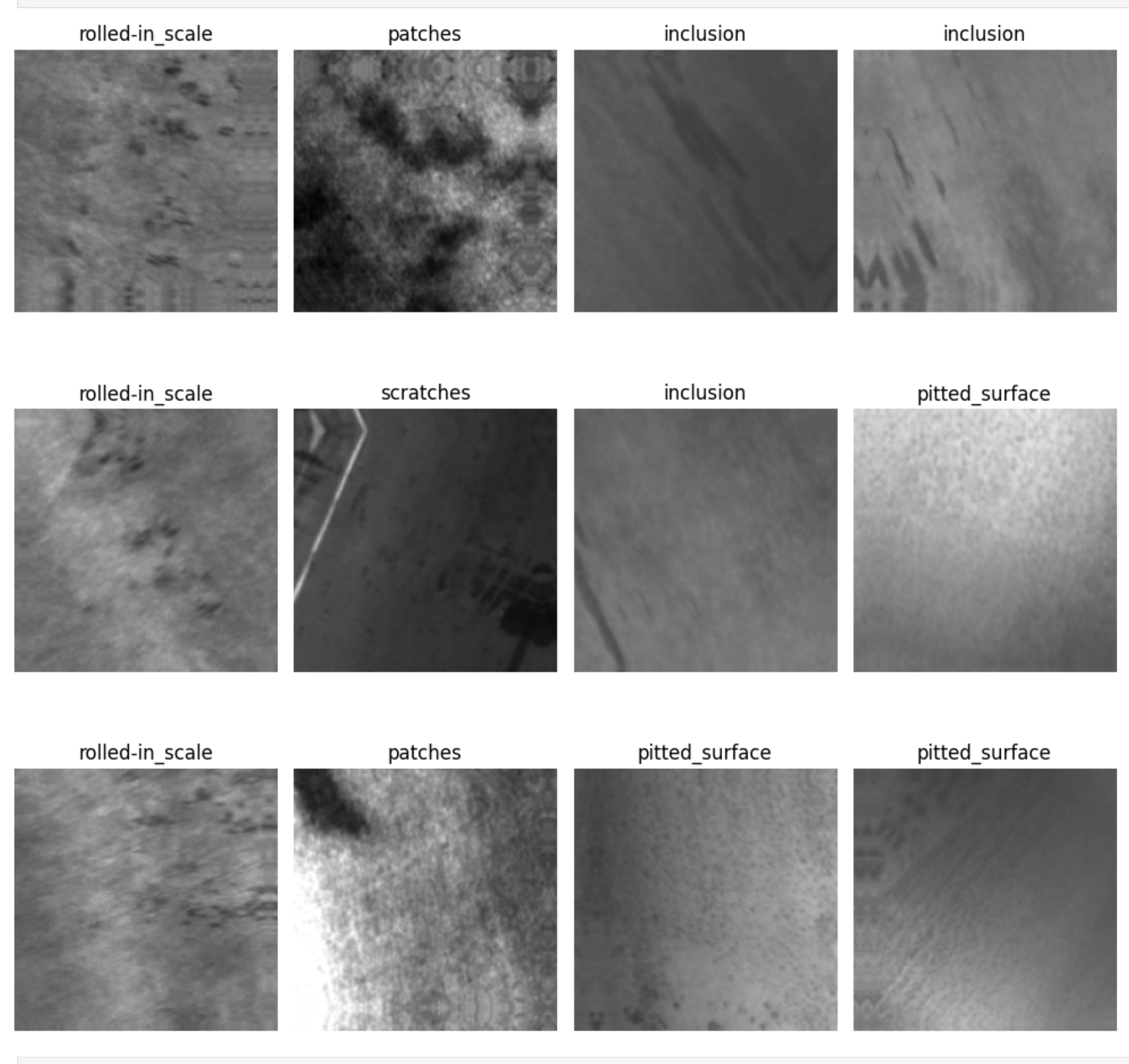
In [ ]:

```python
# Exploratory data analysis – sample images
image_batch, label_batch = next(iter(train_ds_raw))

plt.figure(figsize=(10, 10))
for i in range(12):
    ax = plt.subplot(3, 4, i + 1)
    plt.imshow(image_batch[i].numpy().astype("uint8"), cmap="gray")
    idx = np.argmax(label_batch[i].numpy())
    plt.title(class_names[idx])
    plt.axis("off")
plt.tight_layout()
plt.show()
```


rolled-in_scale, patches, inclusion, inclusion
rolled-in_scale, scratches, inclusion, pitted_surface
rolled-in_scale, patches, pitted_surface, pitted_surface

In [ ]:

```python
# Data augmentation pipeline
data_augmentation = keras.Sequential(
    [
        layers.RandomFlip("horizontal_and_vertical"),
        layers.RandomRotation(0.15),
        layers.RandomZoom(0.15),
        layers.RandomTranslation(0.1, 0.1),
        layers.RandomContrast(0.1),
    ],
    name="data_augmentation",
)

# Visualize augmented images
augmented = data_augmentation(image_batch)

plt.figure(figsize=(10, 10))
for i in range(12):
    ax = plt.subplot(3, 4, i + 1)
    plt.imshow(augmented[i].numpy().astype("uint8"), cmap="gray")
    idx = np.argmax(label_batch[i].numpy())
    plt.title(class_names[idx])
    plt.axis("off")
plt.tight_layout()
plt.show()
```


rolled-in_scale, patches, inclusion, inclusion
rolled-in_scale, scratches, inclusion, pitted_surface
rolled-in_scale, patches, pitted_surface, pitted_surface

In [ ]:

```python
# Convert a tf.data.Dataset to NumPy arrays (for sklearn baselines)

def dataset_to_numpy(ds: tf.data.Dataset) -> Tuple[np.ndarray, np.ndarray]:
    images = []
    labels = []
    for batch_imgs, batch_labels in ds:
        images.append(batch_imgs.numpy())
        labels.append(batch_labels.numpy())
    images = np.concatenate(images, axis=0)
    labels = np.concatenate(labels, axis=0)
    return images, labels

X_train_np, y_train_np = dataset_to_numpy(train_ds_raw)
X_val_np,   y_val_np   = dataset_to_numpy(val_ds_raw)
X_test_np,  y_test_np  = dataset_to_numpy(test_ds_raw)

print("Train np:", X_train_np.shape, y_train_np.shape)
print("Val np:  ", X_val_np.shape,   y_val_np.shape)
print("Test np: ", X_test_np.shape,  y_test_np.shape)

# Flatten for sklearn baselines
X_train_flat = X_train_np.reshape(len(X_train_np), -1)
X_val_flat   = X_val_np.reshape(len(X_val_np),   -1)
X_test_flat  = X_test_np.reshape(len(X_test_np), -1)

y_train_int = np.argmax(y_train_np, axis=1)
y_val_int   = np.argmax(y_val_np,   axis=1)
y_test_int  = np.argmax(y_test_np,  axis=1)
```

```
2025-11-09 21:40:36.124279: W tensorflow/core/framework/local_rendezvous.cc:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
Train np: (1152, 200, 200, 3) (1152, 6)
Val np:   (288, 200, 200, 3) (288, 6)
Test np:  (360, 200, 200, 3) (360, 6)
2025-11-09 21:40:36.364627: W tensorflow/core/framework/local_rendezvous.cc:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
2025-11-09 21:40:36.515691: W tensorflow/core/framework/local_rendezvous.cc:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
```

In [ ]:

```python
# Baseline model 1 – Logistic Regression (flattened pixels)
log_reg = make_pipeline(
    StandardScaler(),
    LogisticRegression(
        max_iter=1000,
        n_jobs=-1,
        verbose=1
    )
)

log_reg.fit(X_train_flat, y_train_int)

print("LogReg – Validation accuracy:", log_reg.score(X_val_flat, y_val_int))
print("LogReg – Test accuracy:      ", log_reg.score(X_test_flat, y_test_int))

y_pred_logreg = log_reg.predict(X_test_flat)
print("\nClassification report – Logistic Regression:\n")
print(classification_report(y_test_int, y_pred_logreg, target_names=class_names))
```

```
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
RUNNING THE L-BFGS-B CODE

           * * *

Machine precision = 2.220D-16
 N =       720006     M =           10

At X0         0 variables are exactly at the bounds

At iterate    0    f=  1.79176D+00    |proj g|=  1.97976D-01
 This problem is unconstrained.
At iterate   50    f=  4.00281D-01    |proj g|=  2.27435D-02

At iterate  100    f=  7.44369D-02    |proj g|=  5.57523D-03

At iterate  150    f=  2.49880D-02    |proj g|=  8.51453D-04

At iterate  200    f=  1.94385D-02    |proj g|=  9.35709D-04

At iterate  250    f=  1.52068D-02    |proj g|=  4.18838D-04

At iterate  300    f=  1.22429D-02    |proj g|=  1.64038D-04

           * * *

Tit   = total number of iterations
Tnf   = total number of function evaluations
Tnint = total number of segments explored during Cauchy searches
Skip  = number of BFGS updates skipped
Nact  = number of active bounds at final generalized Cauchy point
Projg = norm of the final projected gradient
F     = final function value

           * * *

   N    Tit     Tnf  Tnint  Skip  Nact     Projg        F
*****    340    376      1     0     0   8.050D-05   1.081D-02
  F =   1.0812136970376824E-002

CONVERGENCE: NORM_OF_PROJECTED_GRADIENT_<=_PGTOL
LogReg - Validation accuracy: 0.84375
LogReg - Test accuracy:       0.35555555555555557

Classification report - Logistic Regression:

                 precision    recall  f1-score   support

        crazing       0.20      0.03      0.06        60
      inclusion       0.24      0.57      0.34        60
        patches       0.70      0.55      0.62        60
 pitted_surface       0.44      0.57      0.49        60
rolled-in_scale       0.12      0.05      0.07        60
      scratches       0.37      0.37      0.37        60

       accuracy                           0.36       360
      macro avg       0.34      0.36      0.32       360
   weighted avg       0.34      0.36      0.32       360
```

In [ ]:

```python
# Baseline model 2 – Shallow MLP (flattened pixels)
mlp = make_pipeline(
    StandardScaler(),
    MLPClassifier(
        hidden_layer_sizes=(256,),
        activation="relu",
        solver="adam",
        max_iter=100,
        verbose=True
    )
)

mlp.fit(X_train_flat, y_train_int)

print("MLP – Validation accuracy:", mlp.score(X_val_flat, y_val_int))
print("MLP – Test accuracy:      ", mlp.score(X_test_flat, y_test_int))

y_pred_mlp = mlp.predict(X_test_flat)
print("\nClassification report – MLP:\n")
print(classification_report(y_test_int, y_pred_mlp, target_names=class_names))
```

```
Iteration 1, loss = 9.59860216
Iteration 2, loss = 9.78606249
Iteration 3, loss = 10.31794038
Iteration 4, loss = 9.04723517
Iteration 5, loss = 8.77455429
Iteration 6, loss = 7.36266329
Iteration 7, loss = 7.15499241
Iteration 8, loss = 7.03424604
Iteration 9, loss = 5.78534659
Iteration 10, loss = 4.86798903
Iteration 11, loss = 4.91349131
Iteration 12, loss = 4.95179759
Iteration 13, loss = 5.39408512
Iteration 14, loss = 4.89818949
Iteration 15, loss = 4.34715654
Iteration 16, loss = 4.75716776
Iteration 17, loss = 5.28337903
Iteration 18, loss = 4.76200570
Iteration 19, loss = 5.31068160
Iteration 20, loss = 4.55965180
Iteration 21, loss = 4.11457096
Iteration 22, loss = 2.74034188
Iteration 23, loss = 2.99923395
Iteration 24, loss = 2.84963455
Iteration 25, loss = 2.74259680
Iteration 26, loss = 1.95510728
Iteration 27, loss = 2.66155871
Iteration 28, loss = 2.92389298
Iteration 29, loss = 2.41627005
Iteration 30, loss = 2.00154932
Iteration 31, loss = 3.06262974
Iteration 32, loss = 3.09780970
Iteration 33, loss = 3.65458960
Iteration 34, loss = 3.47826326
Iteration 35, loss = 2.94997164
Iteration 36, loss = 3.42325252
Iteration 37, loss = 3.52857798
Training loss did not improve more than tol=0.000100 for 10 consecutive epochs. Stopping.
MLP - Validation accuracy: 0.4409722222222222
MLP - Test accuracy:       0.4111111111111111

Classification report - MLP:

                 precision    recall  f1-score   support

        crazing       0.47      0.15      0.23        60
      inclusion       0.19      0.35      0.25        60
        patches       0.77      0.72      0.74        60
 pitted_surface       0.39      0.85      0.53        60
rolled-in_scale       0.51      0.30      0.38        60
      scratches       0.75      0.10      0.18        60

       accuracy                           0.41       360
      macro avg       0.51      0.41      0.38       360
   weighted avg       0.51      0.41      0.38       360
```

In [ ]:

```python
# Helper function – plot training history
def plot_history(hist):
    acc = hist.history.get("accuracy", [])
    val_acc = hist.history.get("val_accuracy", [])
    loss = hist.history.get("loss", [])
    val_loss = hist.history.get("val_loss", [])
    epochs = range(1, len(acc) + 1)

    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.plot(epochs, acc, label="Train acc")
    plt.plot(epochs, val_acc, label="Val acc")
    plt.xlabel("Epoch")
    plt.ylabel("Accuracy")
    plt.legend()

    plt.subplot(1, 2, 2)
    plt.plot(epochs, loss, label="Train loss")
    plt.plot(epochs, val_loss, label="Val loss")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.legend()

    plt.tight_layout()
    plt.show()
```

In [ ]:

```python
# Custom CNN v1 – baseline convolutional model
def build_cnn_model_v1(
    input_shape=(IMG_HEIGHT, IMG_WIDTH, 3),
    num_classes=NUM_CLASSES
):
    inputs = keras.Input(shape=input_shape)
    x = layers.Rescaling(1.0 / 255)(inputs)

    x = layers.Conv2D(32, 3, padding="same", activation="relu")(x)
    x = layers.MaxPooling2D(2)(x)

    x = layers.Conv2D(64, 3, padding="same", activation="relu")(x)
    x = layers.MaxPooling2D(2)(x)

    x = layers.Conv2D(128, 3, padding="same", activation="relu")(x)
    x = layers.MaxPooling2D(2)(x)

    x = layers.Flatten()(x)
    x = layers.Dense(128, activation="relu")(x)
    outputs = layers.Dense(num_classes, activation="softmax")(x)

    return keras.Model(inputs, outputs, name="cnn_v1")

cnn_v1 = build_cnn_model_v1()
cnn_v1.summary()
```

Model: "cnn_v1"

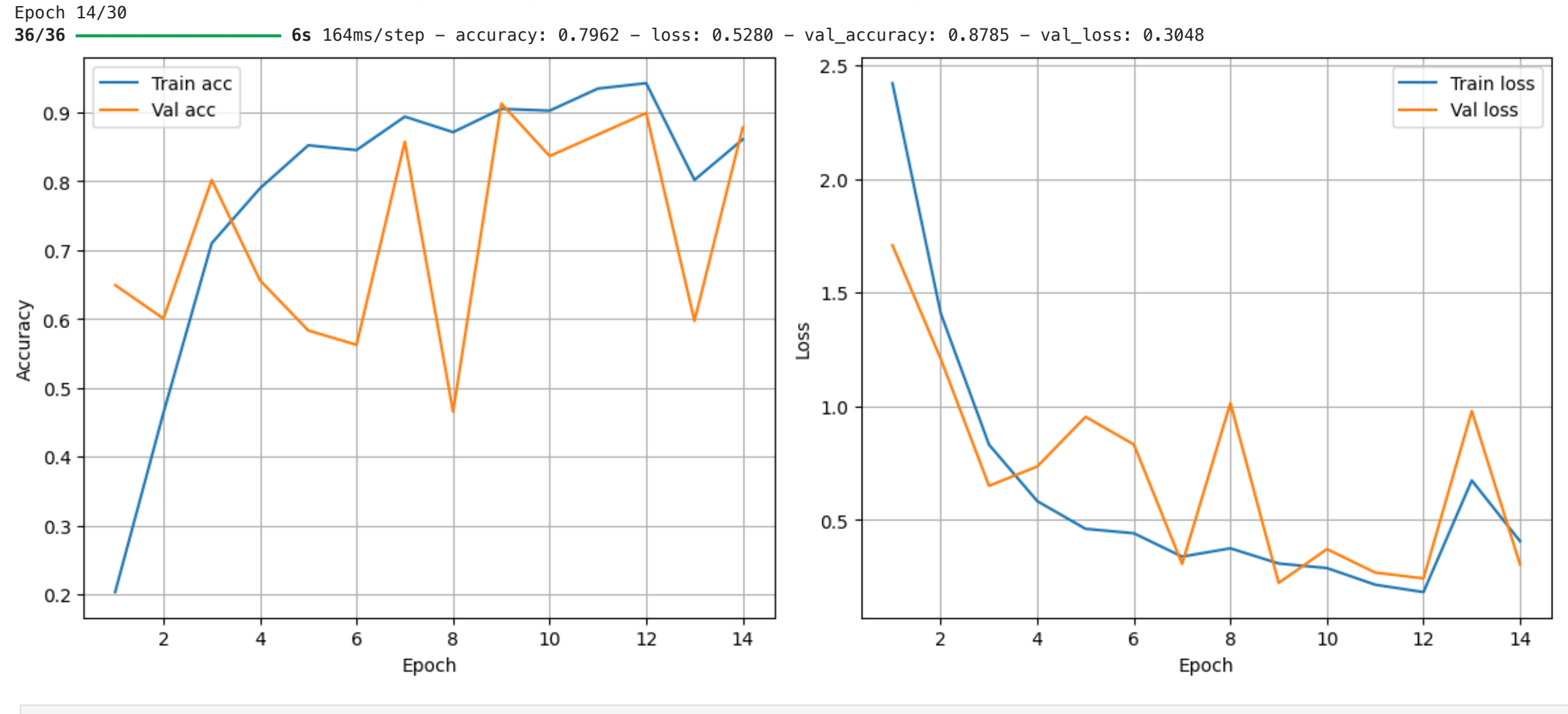
Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 200, 200, 3)	0
rescaling (Rescaling)	(None, 200, 200, 3)	0
conv2d (Conv2D)	(None, 200, 200, 32)	896
max_pooling2d (MaxPooling2D)	(None, 100, 100, 32)	0
conv2d_1 (Conv2D)	(None, 100, 100, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 50, 50, 64)	0
conv2d_2 (Conv2D)	(None, 50, 50, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 25, 25, 128)	0
flatten (Flatten)	(None, 80000)	0
dense (Dense)	(None, 128)	10,240,128
dense_1 (Dense)	(None, 6)	774

Total params: 10,334,150 (39.42 MB)
Trainable params: 10,334,150 (39.42 MB)
Non-trainable params: 0 (0.00 B)

In [ ]:

```python
# Train CNN v1
cnn_v1.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)

early_stopping_v1 = keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=5,
    restore_best_weights=True,
)

history_v1 = cnn_v1.fit(
    train_ds,
    validation_data=val_ds,
    epochs=30,
    callbacks=[early_stopping_v1],
    verbose=1,
)

plot_history(history_v1)
```

```
Epoch 1/30
2025-11-09 21:54:26.497262: I tensorflow/core/grappler/optimizers/custom_graph_optimizer_registry.cc:117] Plugin optimizer for device_type GPU is enabled.
36/36 ━━━━━━━━━━━━━━━━━━━━ 15s 202ms/step - accuracy: 0.1814 - loss: 3.5220 - val_accuracy: 0.6493 - val_loss: 1.7087
Epoch 2/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 7s 200ms/step - accuracy: 0.4154 - loss: 1.5532 - val_accuracy: 0.6007 - val_loss: 1.2089
Epoch 3/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 7s 207ms/step - accuracy: 0.6704 - loss: 0.8714 - val_accuracy: 0.8021 - val_loss: 0.6506
Epoch 4/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 7s 204ms/step - accuracy: 0.8009 - loss: 0.5856 - val_accuracy: 0.6562 - val_loss: 0.7356
Epoch 5/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 7s 187ms/step - accuracy: 0.8195 - loss: 0.5367 - val_accuracy: 0.5833 - val_loss: 0.9539
Epoch 6/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 7s 187ms/step - accuracy: 0.8791 - loss: 0.3715 - val_accuracy: 0.5625 - val_loss: 0.8322
Epoch 7/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 6s 178ms/step - accuracy: 0.8847 - loss: 0.3631 - val_accuracy: 0.8576 - val_loss: 0.3070
Epoch 8/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 6s 171ms/step - accuracy: 0.8844 - loss: 0.3136 - val_accuracy: 0.4653 - val_loss: 1.0145
Epoch 9/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 6s 170ms/step - accuracy: 0.8783 - loss: 0.3810 - val_accuracy: 0.9132 - val_loss: 0.2237
Epoch 10/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 6s 166ms/step - accuracy: 0.9285 - loss: 0.2267 - val_accuracy: 0.8368 - val_loss: 0.3721
Epoch 11/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 6s 164ms/step - accuracy: 0.9200 - loss: 0.2430 - val_accuracy: 0.8681 - val_loss: 0.2689
Epoch 12/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 6s 166ms/step - accuracy: 0.9475 - loss: 0.1642 - val_accuracy: 0.8993 - val_loss: 0.2433
Epoch 13/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 6s 163ms/step - accuracy: 0.8066 - loss: 0.6367 - val_accuracy: 0.5972 - val_loss: 0.9795
Epoch 14/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 6s 164ms/step - accuracy: 0.7962 - loss: 0.5280 - val_accuracy: 0.8785 - val_loss: 0.3048
```



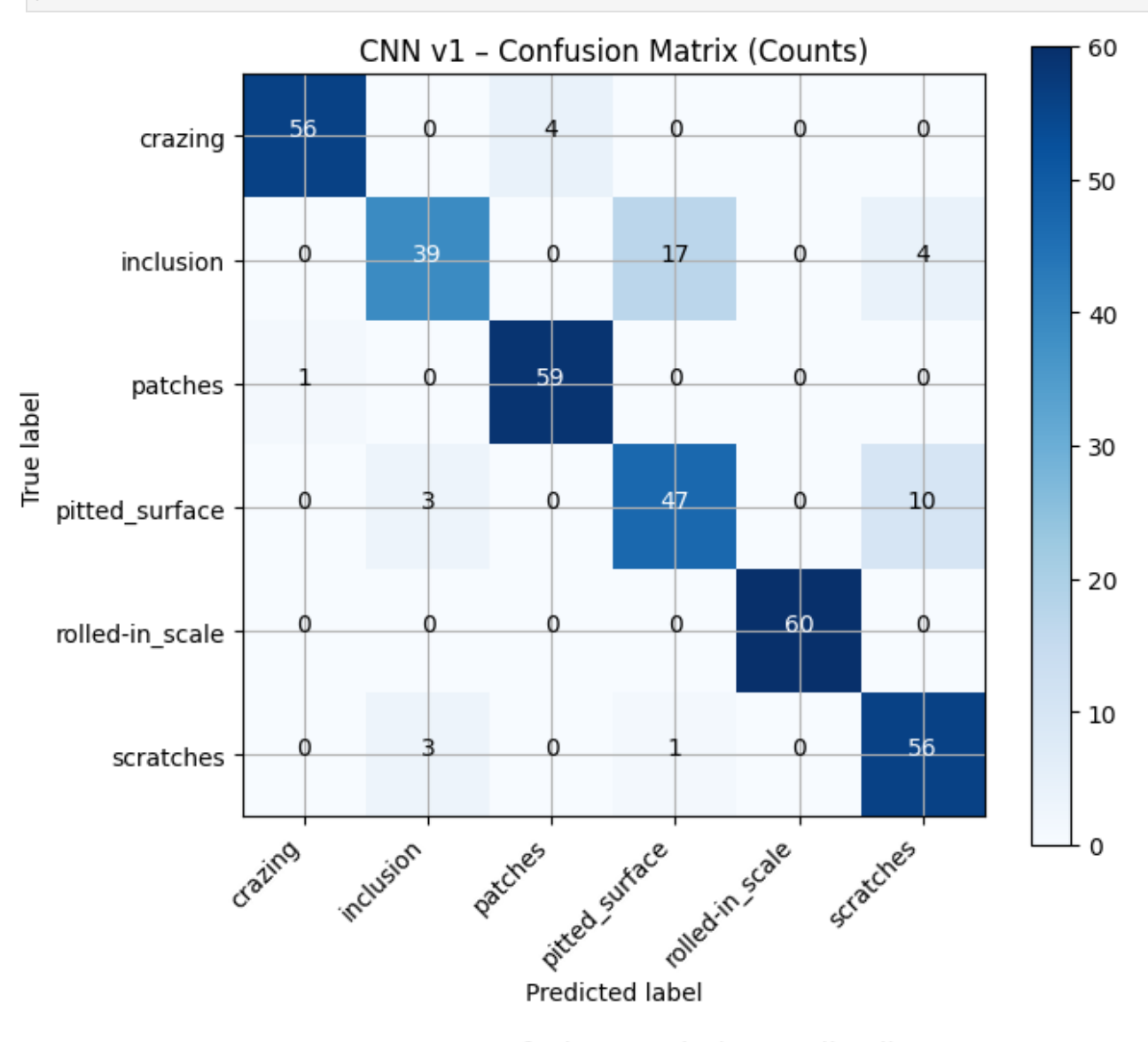
In [ ]:

```python
# Evaluate CNN v1
test_loss_v1, test_acc_v1 = cnn_v1.evaluate(test_ds, verbose=0)
print(f"CNN v1 – Test loss: {test_loss_v1:.4f}")
print(f"CNN v1 – Test accuracy: {test_acc_v1:.4f}")

def get_predictions_and_labels(ds, model):
    y_true = []
    y_pred = []
    for batch_imgs, batch_labels in ds:
        preds = model.predict(batch_imgs, verbose=0)
        y_true.append(np.argmax(batch_labels.numpy(), axis=1))
        y_pred.append(np.argmax(preds, axis=1))
    return np.concatenate(y_true), np.concatenate(y_pred)

y_true_v1, y_pred_v1 = get_predictions_and_labels(test_ds, cnn_v1)

print("\nClassification report – CNN v1:\n")
print(classification_report(y_true_v1, y_pred_v1, target_names=class_names))

cm_v1 = confusion_matrix(y_true_v1, y_pred_v1)
print("Confusion matrix – CNN v1:\n", cm_v1)
```

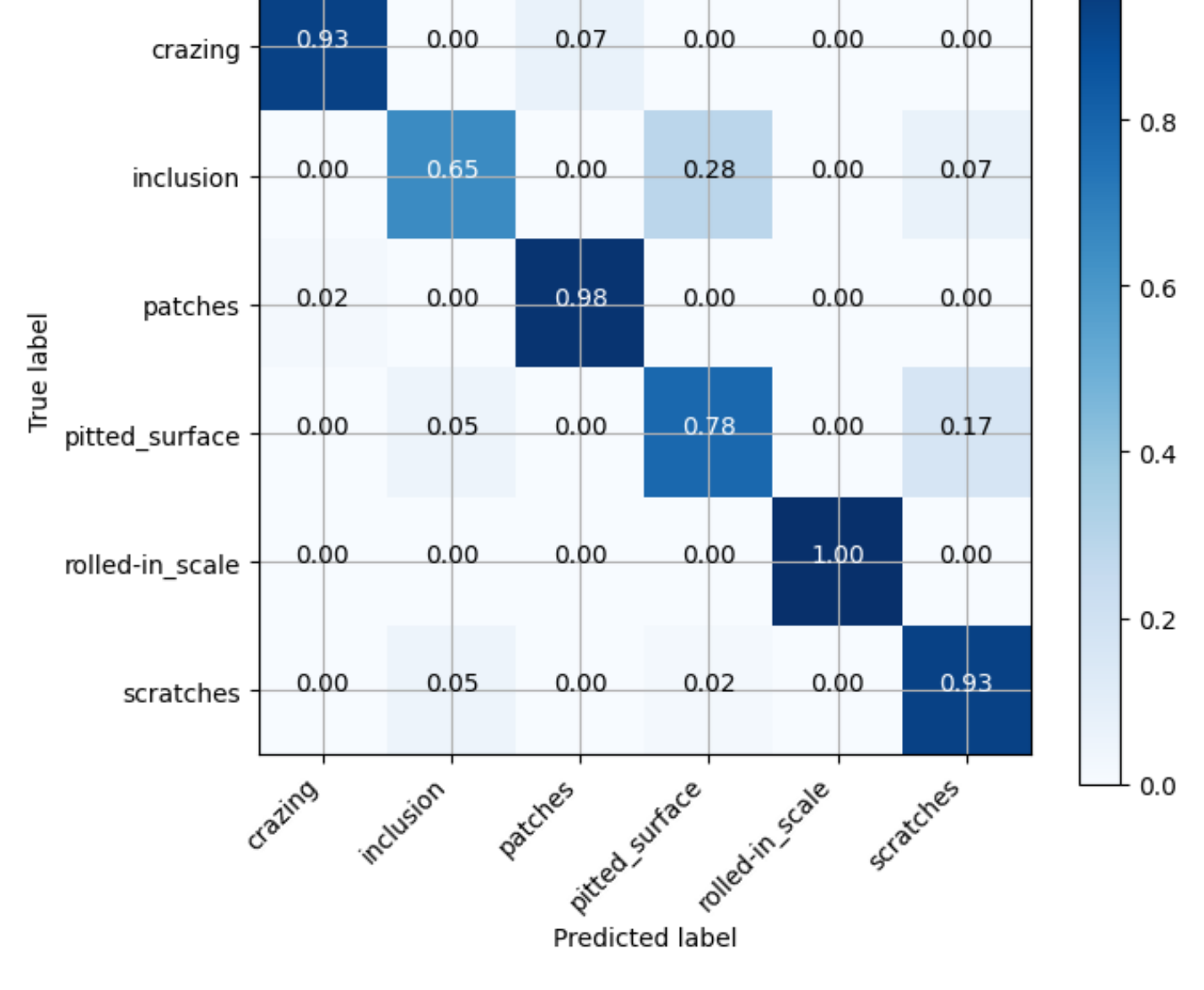
```
CNN v1 - Test loss: 0.4197
CNN v1 - Test accuracy: 0.8806

Classification report - CNN v1:

                 precision    recall  f1-score   support

        crazing       0.98      0.93      0.96        60
      inclusion       0.87      0.65      0.74        60
        patches       0.94      0.98      0.96        60
 pitted_surface       0.72      0.78      0.75        60
rolled-in_scale       1.00      1.00      1.00        60
      scratches       0.80      0.93      0.86        60

       accuracy                           0.88       360
      macro avg       0.88      0.88      0.88       360
   weighted avg       0.88      0.88      0.88       360

Confusion matrix - CNN v1:
 [[56  0  4  0  0  0]
 [ 0 39  0 17  0  4]
 [ 1  0 59  0  0  0]
 [ 0  3  0 47  0 10]
 [ 0  0  0  0 60  0]
 [ 0  3  0  1  0 56]]
2025-11-09 21:56:08.766682: W tensorflow/core/framework/local_rendezvous.cc:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
```

In [ ]:

```python
# Helper – confusion matrix plotting
import itertools

def plot_confusion_matrix(cm, classes, normalize=False, title="Confusion matrix"):
    if normalize:
        cm = cm.astype("float") / cm.sum(axis=1, keepdims=True)

    plt.figure(figsize=(7, 6))
    plt.imshow(cm, interpolation="nearest", cmap="Blues")
    plt.title(title)
    plt.colorbar()
    tick_marks = np.arange(len(classes))
    plt.xticks(tick_marks, classes, rotation=45, ha="right")
    plt.yticks(tick_marks, classes)

    fmt = ".2f" if normalize else "d"
    thresh = cm.max() / 2.0
    for i, j in itertools.product(range(cm.shape[0]), range(cm.shape[1])):
        plt.text(
            j, i, format(cm[i, j], fmt),
            horizontalalignment="center",
            color="white" if cm[i, j] > thresh else "black",
        )

    plt.ylabel("True label")
    plt.xlabel("Predicted label")
    plt.tight_layout()
    plt.show()

plot_confusion_matrix(cm_v1, class_names, normalize=False, title="CNN v1 – Confusion Matrix (Counts)")
plot_confusion_matrix(cm_v1, class_names, normalize=True,  title="CNN v1 – Confusion Matrix (Normalized)")
```


CNN v1 – Confusion Matrix (Counts)

CNN v1 – Confusion Matrix (Normalized)



```
In [ ]: # Custom CNN v2 – with data augmentation, BatchNorm, and L2
def build_cnn_model_v2(
    input_shape=(IMG_HEIGHT, IMG_WIDTH, 3),
    num_classes=NUM_CLASSES,
    l2_reg=1e-4
):
    inputs = keras.Input(shape=input_shape)

    x = data_augmentation(inputs)
    x = layers.Rescaling(1.0 / 255)(x)

    x = layers.Conv2D(32, 3, padding="same", activation="relu",
                      kernel_regularizer=regularizers.l2(l2_reg))(x)
    x = layers.BatchNormalization()(x)
    x = layers.MaxPooling2D(2)(x)
    x = layers.Dropout(0.2)(x)

    x = layers.Conv2D(64, 3, padding="same", activation="relu",
                      kernel_regularizer=regularizers.l2(l2_reg))(x)
    x = layers.BatchNormalization()(x)
    x = layers.MaxPooling2D(2)(x)
    x = layers.Dropout(0.3)(x)

    x = layers.Conv2D(128, 3, padding="same", activation="relu",
                      kernel_regularizer=regularizers.l2(l2_reg))(x)
    x = layers.BatchNormalization()(x)
    x = layers.MaxPooling2D(2)(x)
    x = layers.Dropout(0.4)(x)

    x = layers.Flatten()(x)
    x = layers.Dense(128, activation="relu",
                     kernel_regularizer=regularizers.l2(l2_reg))(x)
    x = layers.Dropout(0.4)(x)
    outputs = layers.Dense(num_classes, activation="softmax")(x)

    return keras.Model(inputs, outputs, name="cnn_v2")

cnn_v2 = build_cnn_model_v2()
cnn_v2.summary()
```

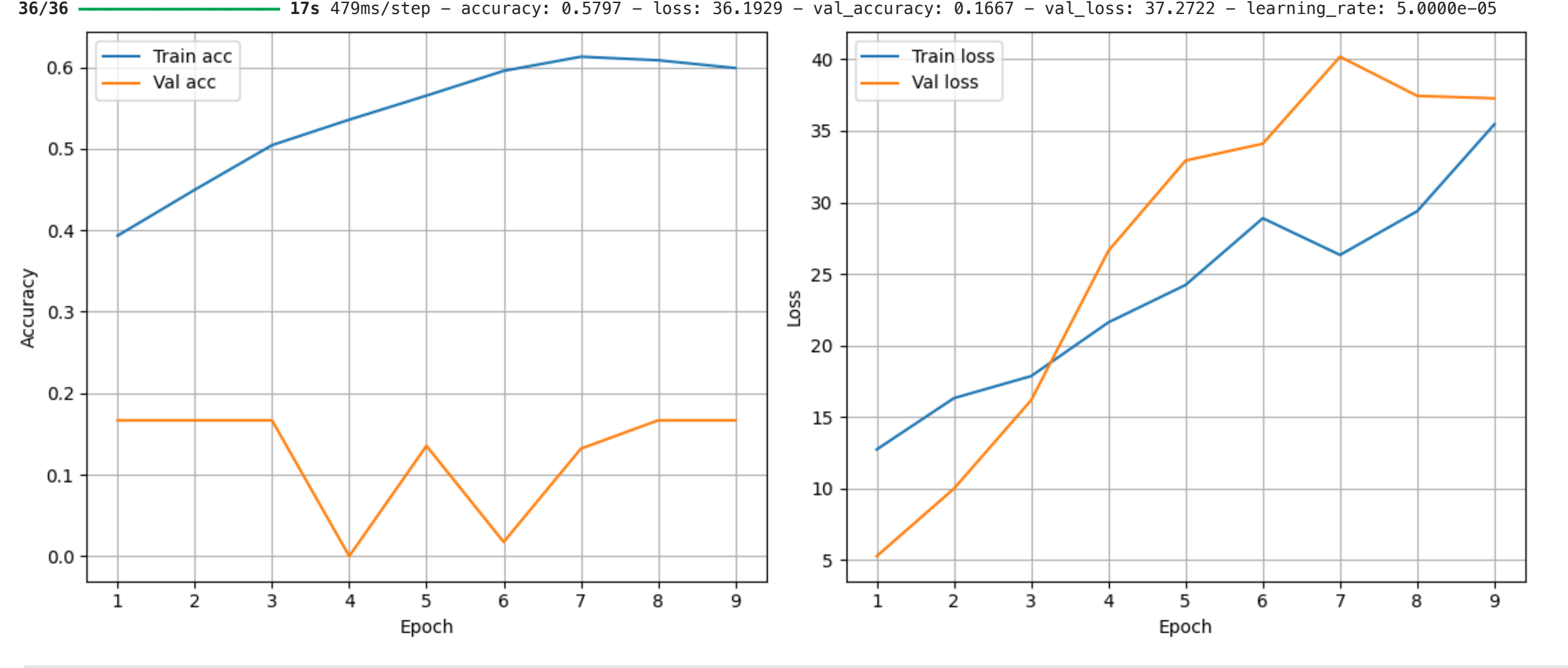
Model: "cnn_v2"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 200, 200, 3)	0
data_augmentation (Sequential)	(None, 200, 200, 3)	0
rescaling_1 (Rescaling)	(None, 200, 200, 3)	0
conv2d_3 (Conv2D)	(None, 200, 200, 32)	896
batch_normalization (BatchNormalization)	(None, 200, 200, 32)	128
max_pooling2d_3 (MaxPooling2D)	(None, 100, 100, 32)	0
dropout (Dropout)	(None, 100, 100, 32)	0
conv2d_4 (Conv2D)	(None, 100, 100, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 100, 100, 64)	256
max_pooling2d_4 (MaxPooling2D)	(None, 50, 50, 64)	0
dropout_1 (Dropout)	(None, 50, 50, 64)	0
conv2d_5 (Conv2D)	(None, 50, 50, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 50, 50, 128)	512
max_pooling2d_5 (MaxPooling2D)	(None, 25, 25, 128)	0
dropout_2 (Dropout)	(None, 25, 25, 128)	0
flatten_1 (Flatten)	(None, 80000)	0
dense_2 (Dense)	(None, 128)	10,240,128
dropout_3 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 6)	774

Total params: 10,335,046 (39.43 MB)
Trainable params: 10,334,598 (39.42 MB)
Non-trainable params: 448 (1.75 KB)

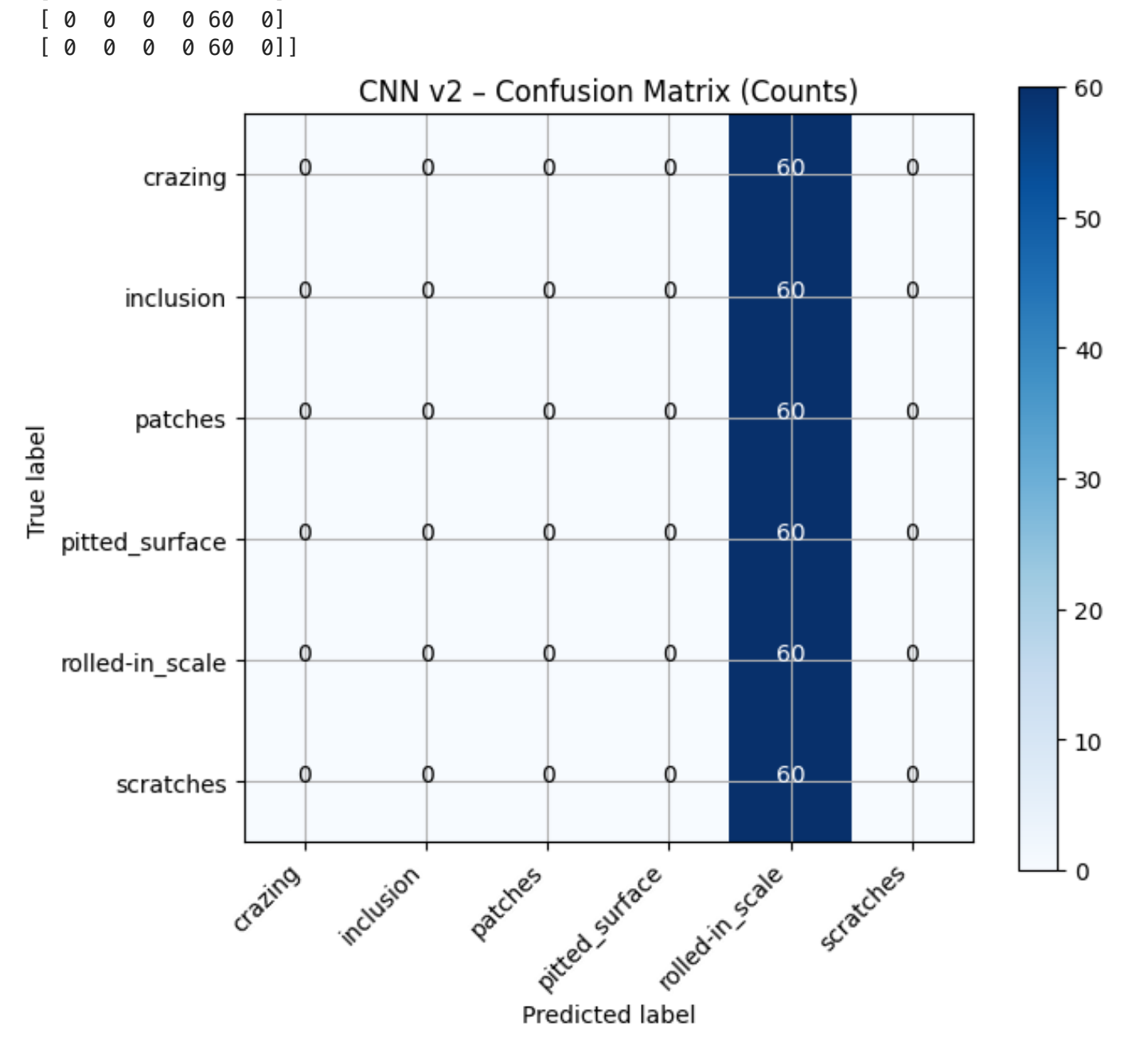
```
In [ ]: # Compute class weights to handle any class imbalance
class_weights_array = class_weight.compute_class_weight(
    class_weight="balanced",
    classes=np.unique(y_train_int),
    y=y_train_int,
)
class_weights = dict(enumerate(class_weights_array))
print("Class weights:", class_weights)

# Train CNN v2
cnn_v2.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-4),
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)

early_stopping_v2 = keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=8,
    restore_best_weights=True,
)

reduce_lr_v2 = keras.callbacks.ReduceLROnPlateau(
    monitor="val_loss",
    factor=0.5,
    patience=4,
    verbose=1,
)

history_v2 = cnn_v2.fit(
    train_ds,
    validation_data=val_ds,
    epochs=50,
    class_weight=class_weights,
    callbacks=[early_stopping_v2, reduce_lr_v2],
    verbose=1,
)

plot_history(history_v2)
```

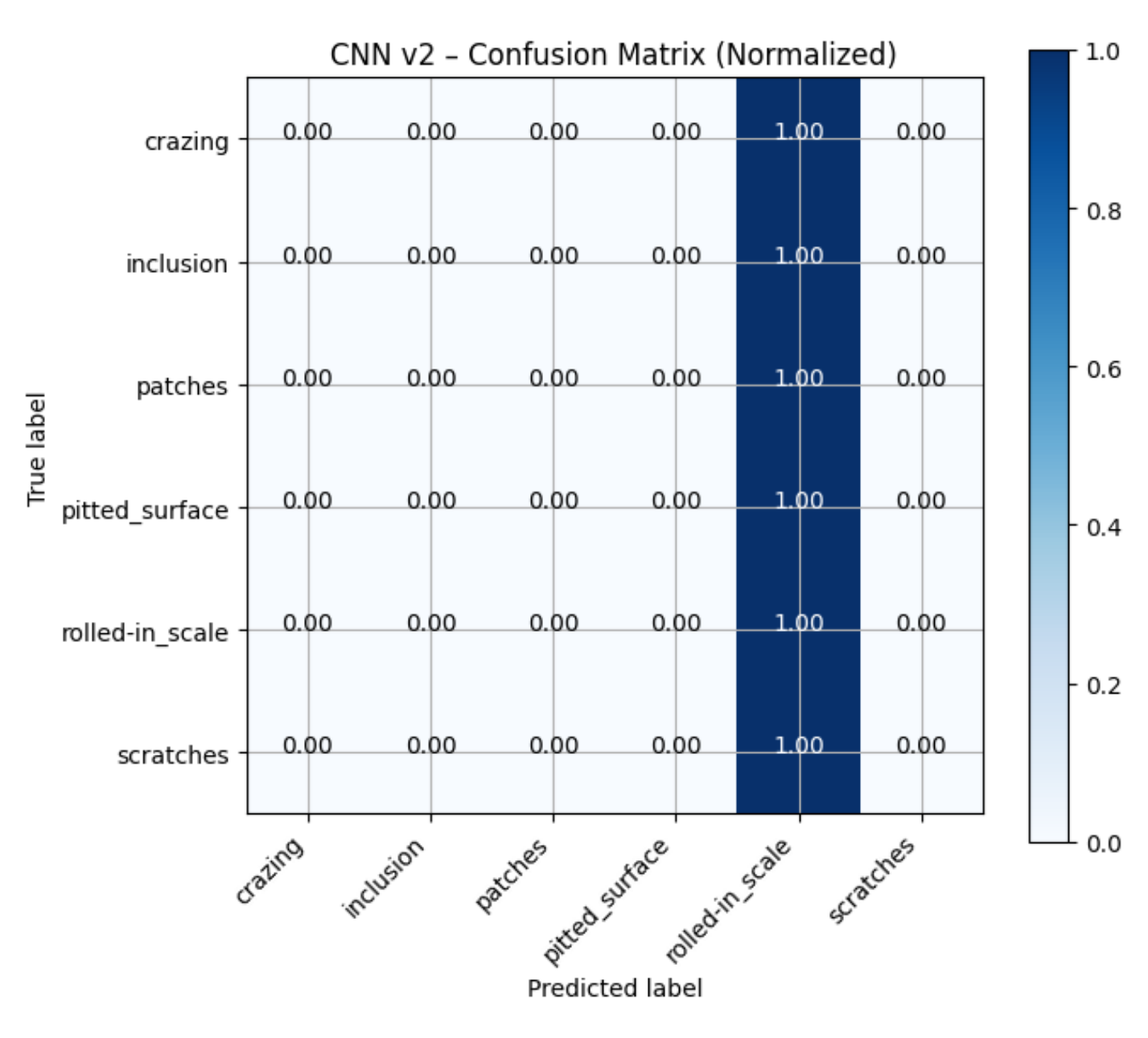
```
Class weights: {0: 0.9846153846153847, 1: 0.9696969696969697, 2: 0.9896907216494846, 3: 0.9948186528497409, 4: 1.0052356020942408, 5: 1.0607734806629834}
Epoch 1/50
36/36 ━━━━━━━━━━━━━━━━━━━━ 27s 501ms/step - accuracy: 0.3414 - loss: 10.7902 - val_accuracy: 0.1667 - val_loss: 5.2829 - learning_rate: 1.0000e-04
Epoch 2/50
36/36 ━━━━━━━━━━━━━━━━━━━━ 18s 493ms/step - accuracy: 0.4368 - loss: 16.0251 - val_accuracy: 0.1667 - val_loss: 10.0014 - learning_rate: 1.0000e-04
Epoch 3/50
36/36 ━━━━━━━━━━━━━━━━━━━━ 18s 486ms/step - accuracy: 0.4934 - loss: 17.1134 - val_accuracy: 0.1667 - val_loss: 16.1817 - learning_rate: 1.0000e-04
Epoch 4/50
36/36 ━━━━━━━━━━━━━━━━━━━━ 18s 488ms/step - accuracy: 0.5441 - loss: 19.3819 - val_accuracy: 0.0000e+00 - val_loss: 26.6145 - learning_rate: 1.0000e-04
Epoch 5/50
36/36 ━━━━━━━━━━━━━━━━━━━━ 0s 457ms/step - accuracy: 0.5699 - loss: 21.9484
Epoch 5: ReduceLROnPlateau reducing learning rate to 4.999999873689376e-05.
36/36 ━━━━━━━━━━━━━━━━━━━━ 17s 477ms/step - accuracy: 0.5697 - loss: 22.0102 - val_accuracy: 0.1354 - val_loss: 32.9255 - learning_rate: 1.0000e-04
Epoch 6/50
36/36 ━━━━━━━━━━━━━━━━━━━━ 17s 480ms/step - accuracy: 0.5810 - loss: 28.7947 - val_accuracy: 0.0174 - val_loss: 34.1053 - learning_rate: 5.0000e-05
Epoch 7/50
36/36 ━━━━━━━━━━━━━━━━━━━━ 18s 505ms/step - accuracy: 0.6277 - loss: 24.8507 - val_accuracy: 0.1319 - val_loss: 40.1812 - learning_rate: 5.0000e-05
Epoch 8/50
36/36 ━━━━━━━━━━━━━━━━━━━━ 17s 480ms/step - accuracy: 0.6017 - loss: 29.1062 - val_accuracy: 0.1667 - val_loss: 37.4444 - learning_rate: 5.0000e-05
Epoch 9/50
36/36 ━━━━━━━━━━━━━━━━━━━━ 0s 460ms/step - accuracy: 0.5792 - loss: 36.2131
Epoch 9: ReduceLROnPlateau reducing learning rate to 2.499999936844688e-05.
36/36 ━━━━━━━━━━━━━━━━━━━━ 17s 479ms/step - accuracy: 0.5797 - loss: 36.1929 - val_accuracy: 0.1667 - val_loss: 37.2722 - learning_rate: 5.0000e-05
```



```
In [ ]: # Evaluate CNN v2
test_loss_v2, test_acc_v2 = cnn_v2.evaluate(test_ds, verbose=0)
print(f"CNN v2 – Test loss: {test_loss_v2:.4f}")
print(f"CNN v2 – Test accuracy: {test_acc_v2:.4f}")

y_true_v2, y_pred_v2 = get_predictions_and_labels(test_ds, cnn_v2)

print("\nClassification report – CNN v2:\n")
print(classification_report(y_true_v2, y_pred_v2, target_names=class_names))

cm_v2 = confusion_matrix(y_true_v2, y_pred_v2)
print("Confusion matrix – CNN v2:\n", cm_v2)

plot_confusion_matrix(cm_v2, class_names, normalize=False, title="CNN v2 – Confusion Matrix (Counts)")
plot_confusion_matrix(cm_v2, class_names, normalize=True,  title="CNN v2 – Confusion Matrix (Normalized)")
```

```
CNN v2 – Test loss: 4.4386
CNN v2 – Test accuracy: 0.1667
2025-11-09 21:59:00.316353: W tensorflow/core/framework/local_rendezvous.cc:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
/Users/magmac/Downloads/.venv/lib/python3.9/site-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/Users/magmac/Downloads/.venv/lib/python3.9/site-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/Users/magmac/Downloads/.venv/lib/python3.9/site-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
Classification report – CNN v2:

                 precision    recall  f1-score   support

        crazing       0.00      0.00      0.00        60
      inclusion       0.00      0.00      0.00        60
        patches       0.00      0.00      0.00        60
 pitted_surface       0.00      0.00      0.00        60
rolled-in_scale       0.17      1.00      0.29        60
      scratches       0.00      0.00      0.00        60

       accuracy                           0.17       360
      macro avg       0.03      0.17      0.05       360
   weighted avg       0.03      0.17      0.05       360

Confusion matrix – CNN v2:
 [[ 0  0  0  0 60  0]
 [ 0  0  0  0 60  0]
 [ 0  0  0  0 60  0]
 [ 0  0  0  0 60  0]
 [ 0  0  0  0 60  0]
 [ 0  0  0  0 60  0]]
```





```
In [ ]: # Transfer learning – EfficientNetB0 feature extractor
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.applications.efficientnet import preprocess_input

def build_transfer_model(
    input_shape=(IMG_HEIGHT, IMG_WIDTH, 3),
    num_classes=NUM_CLASSES,
    dropout_rate=0.4
):
    base_model = EfficientNetB0(
        include_top=False,
        weights="imagenet",
        input_shape=input_shape,
    )
    base_model.trainable = False  # feature extractor

    inputs = keras.Input(shape=input_shape)
    x = data_augmentation(inputs)
    x = preprocess_input(x)
    x = base_model(x, training=False)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dropout(dropout_rate)(x)
    outputs = layers.Dense(num_classes, activation="softmax")(x)

    model = keras.Model(inputs, outputs, name="efficientnet_b0_transfer")
    return model

transfer_model = build_transfer_model()
transfer_model.summary()
```

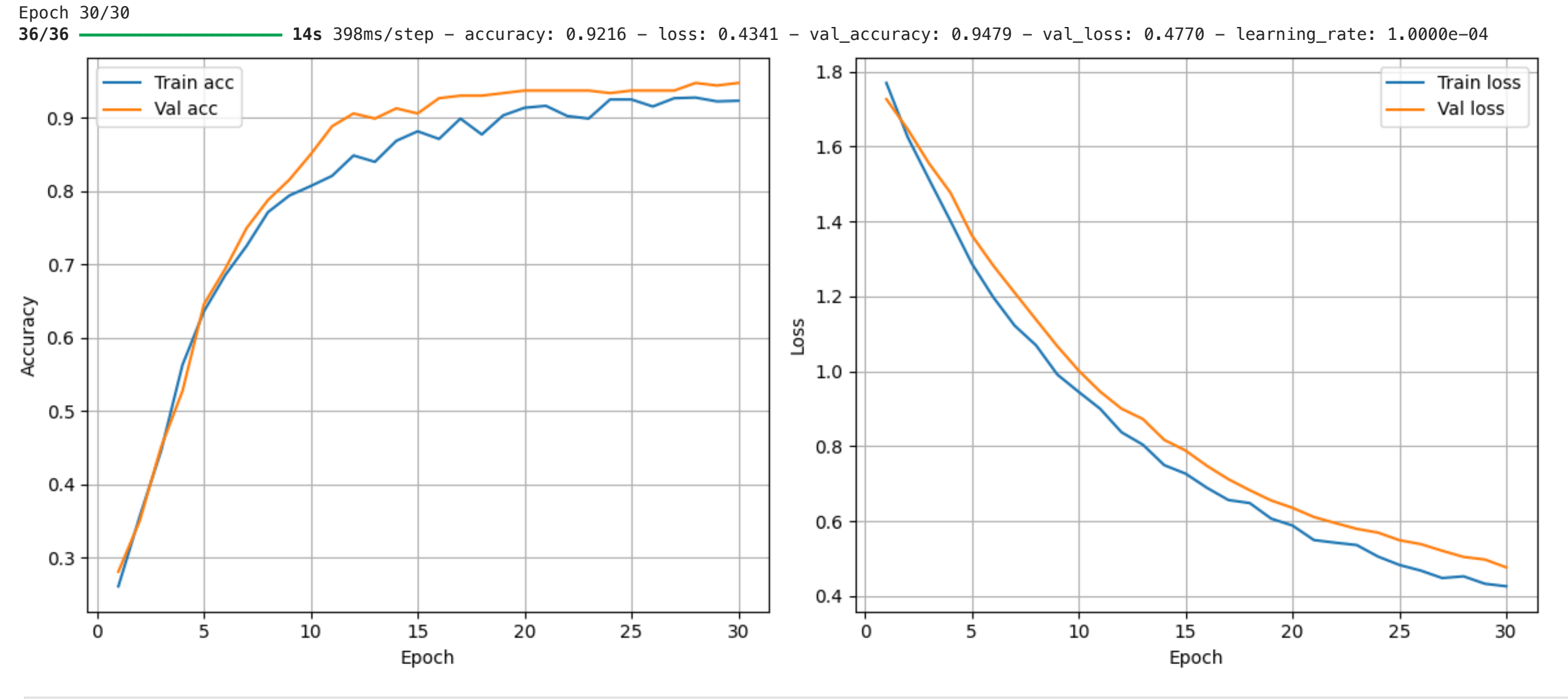
Model: "efficientnet_b0_transfer"

Layer (type)	Output Shape	Param #
input_layer_4 (InputLayer)	(None, 200, 200, 3)	0
data_augmentation (Sequential)	(None, 200, 200, 3)	0
efficientnetb0 (Functional)	(None, 7, 7, 1280)	4,049,571
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout_4 (Dropout)	(None, 1280)	0
dense_4 (Dense)	(None, 6)	7,686

Total params: 4,057,257 (15.48 MB)
Trainable params: 7,686 (30.02 KB)
Non-trainable params: 4,049,571 (15.45 MB)

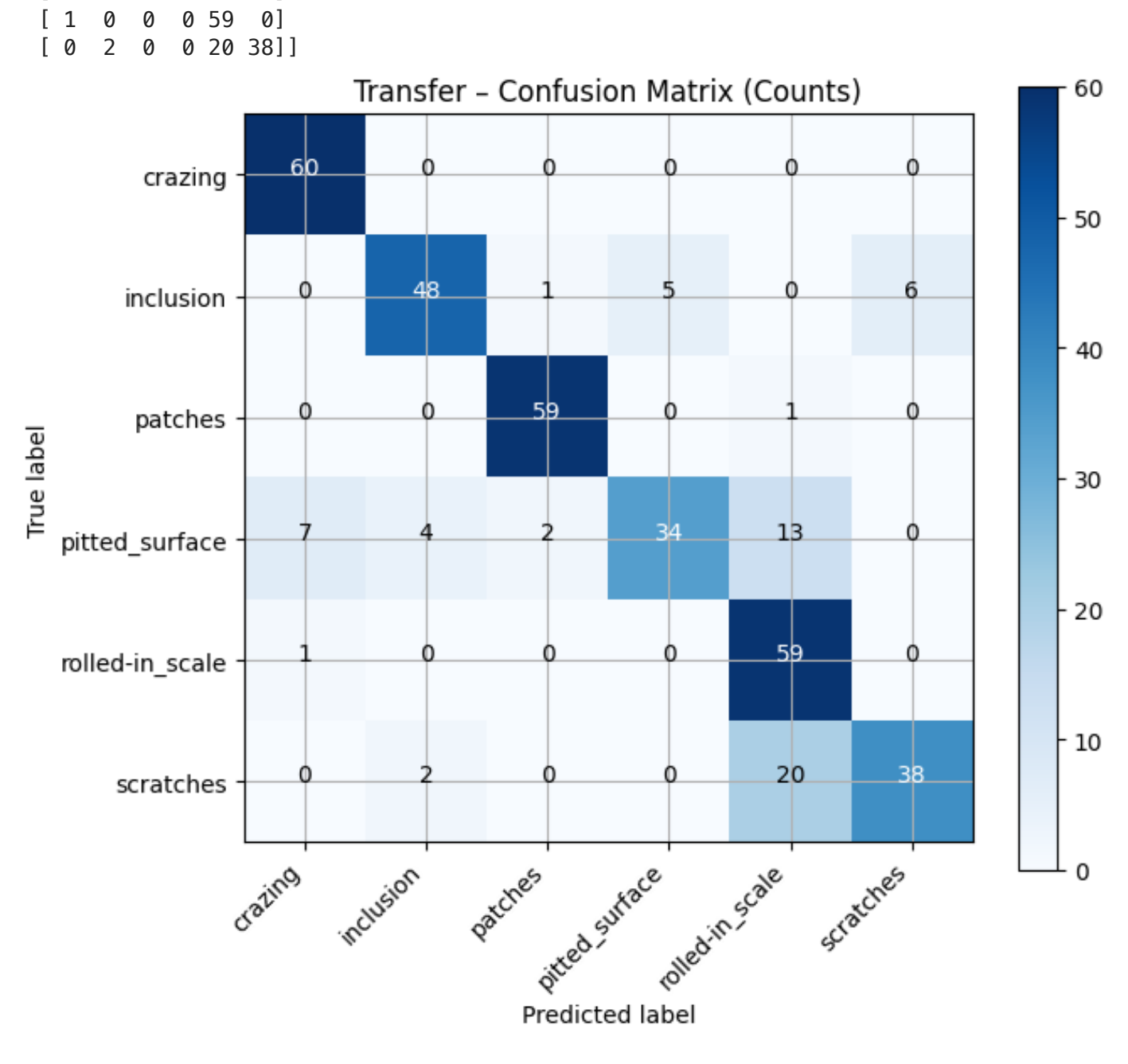
```
In [ ]: # Train transfer learning model
transfer_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-4),
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)

early_stopping_tl = keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=6,
    restore_best_weights=True,
)

reduce_lr_tl = keras.callbacks.ReduceLROnPlateau(
    monitor="val_loss",
    factor=0.5,
    patience=3,
    verbose=1,
)

history_tl = transfer_model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=30,
    callbacks=[early_stopping_tl, reduce_lr_tl],
    verbose=1,
)

plot_history(history_tl)
```

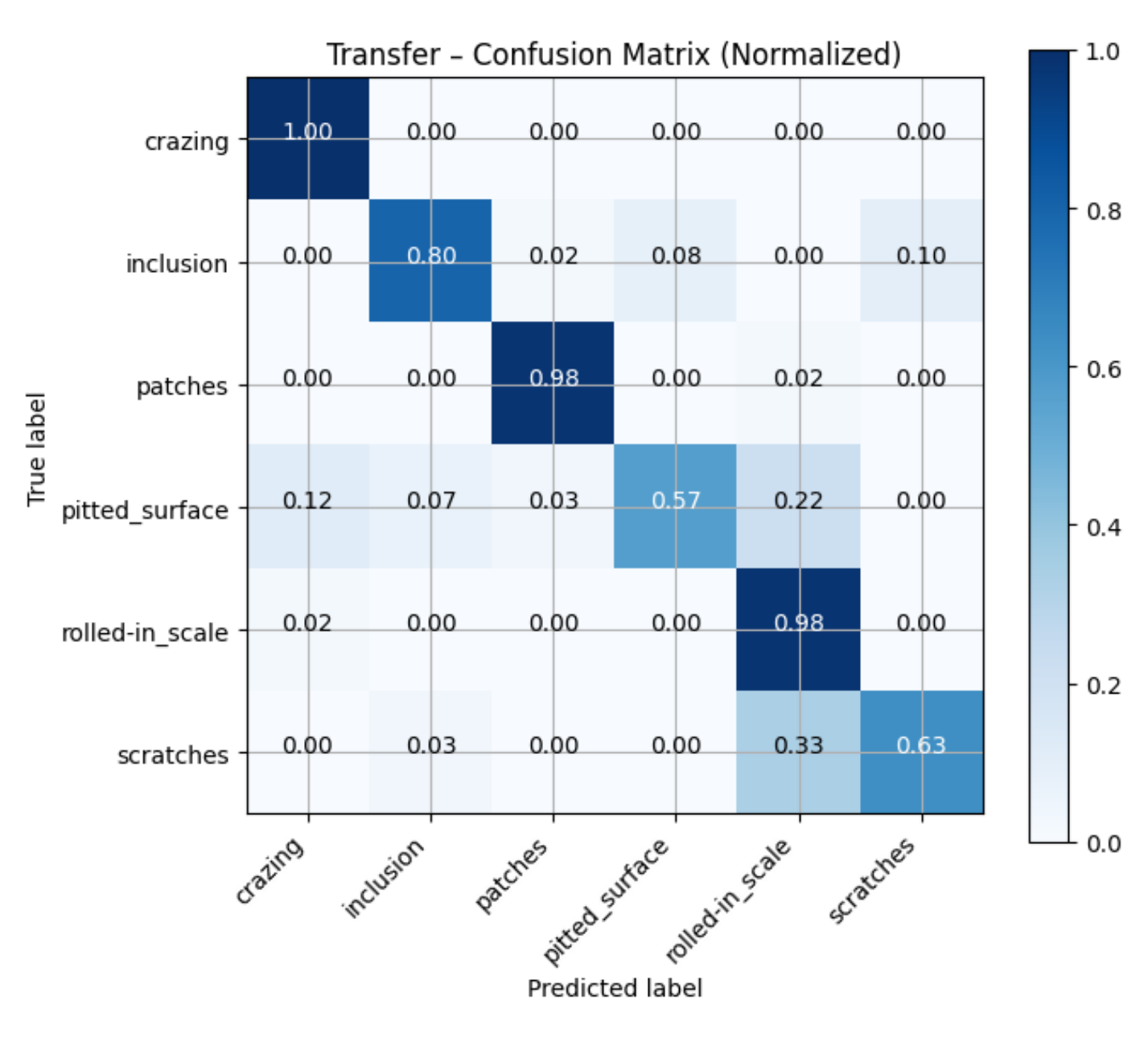
```
Epoch 1/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 38s 591ms/step - accuracy: 0.2378 - loss: 1.7918 - val_accuracy: 0.2812 - val_loss: 1.7262 - learning_rate: 1.0000e-04
Epoch 2/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 16s 461ms/step - accuracy: 0.3426 - loss: 1.6492 - val_accuracy: 0.3507 - val_loss: 1.6456 - learning_rate: 1.0000e-04
Epoch 3/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 15s 430ms/step - accuracy: 0.4295 - loss: 1.5327 - val_accuracy: 0.4514 - val_loss: 1.5543 - learning_rate: 1.0000e-04
Epoch 4/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 15s 425ms/step - accuracy: 0.5671 - loss: 1.3968 - val_accuracy: 0.5278 - val_loss: 1.4764 - learning_rate: 1.0000e-04
Epoch 5/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 15s 407ms/step - accuracy: 0.6174 - loss: 1.2924 - val_accuracy: 0.6458 - val_loss: 1.3627 - learning_rate: 1.0000e-04
Epoch 6/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 14s 404ms/step - accuracy: 0.6676 - loss: 1.2245 - val_accuracy: 0.6944 - val_loss: 1.2822 - learning_rate: 1.0000e-04
Epoch 7/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 14s 397ms/step - accuracy: 0.7191 - loss: 1.1286 - val_accuracy: 0.7500 - val_loss: 1.2099 - learning_rate: 1.0000e-04
Epoch 8/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 14s 399ms/step - accuracy: 0.7618 - loss: 1.0841 - val_accuracy: 0.7882 - val_loss: 1.1381 - learning_rate: 1.0000e-04
Epoch 9/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 14s 391ms/step - accuracy: 0.7708 - loss: 1.0212 - val_accuracy: 0.8160 - val_loss: 1.0669 - learning_rate: 1.0000e-04
Epoch 10/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 14s 399ms/step - accuracy: 0.7878 - loss: 0.9634 - val_accuracy: 0.8507 - val_loss: 1.0018 - learning_rate: 1.0000e-04
Epoch 11/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 14s 397ms/step - accuracy: 0.8092 - loss: 0.9154 - val_accuracy: 0.8889 - val_loss: 0.9459 - learning_rate: 1.0000e-04
Epoch 12/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 15s 406ms/step - accuracy: 0.8354 - loss: 0.8598 - val_accuracy: 0.9062 - val_loss: 0.9003 - learning_rate: 1.0000e-04
Epoch 13/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 14s 399ms/step - accuracy: 0.8244 - loss: 0.8173 - val_accuracy: 0.8993 - val_loss: 0.8727 - learning_rate: 1.0000e-04
Epoch 14/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 14s 405ms/step - accuracy: 0.8652 - loss: 0.7544 - val_accuracy: 0.9132 - val_loss: 0.8170 - learning_rate: 1.0000e-04
Epoch 15/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 14s 404ms/step - accuracy: 0.8623 - loss: 0.7508 - val_accuracy: 0.9062 - val_loss: 0.7885 - learning_rate: 1.0000e-04
Epoch 16/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 14s 399ms/step - accuracy: 0.8704 - loss: 0.6940 - val_accuracy: 0.9271 - val_loss: 0.7477 - learning_rate: 1.0000e-04
Epoch 17/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 14s 398ms/step - accuracy: 0.8877 - loss: 0.6593 - val_accuracy: 0.9306 - val_loss: 0.7122 - learning_rate: 1.0000e-04
Epoch 18/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 14s 402ms/step - accuracy: 0.8715 - loss: 0.6582 - val_accuracy: 0.9306 - val_loss: 0.6827 - learning_rate: 1.0000e-04
Epoch 19/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 14s 401ms/step - accuracy: 0.8879 - loss: 0.6180 - val_accuracy: 0.9340 - val_loss: 0.6555 - learning_rate: 1.0000e-04
Epoch 20/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 14s 398ms/step - accuracy: 0.9166 - loss: 0.5910 - val_accuracy: 0.9375 - val_loss: 0.6358 - learning_rate: 1.0000e-04
Epoch 21/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 15s 403ms/step - accuracy: 0.9229 - loss: 0.5451 - val_accuracy: 0.9375 - val_loss: 0.6114 - learning_rate: 1.0000e-04
Epoch 22/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 14s 399ms/step - accuracy: 0.8844 - loss: 0.5642 - val_accuracy: 0.9375 - val_loss: 0.5953 - learning_rate: 1.0000e-04
Epoch 23/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 15s 406ms/step - accuracy: 0.9026 - loss: 0.5362 - val_accuracy: 0.9375 - val_loss: 0.5798 - learning_rate: 1.0000e-04
Epoch 24/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 14s 397ms/step - accuracy: 0.9155 - loss: 0.5117 - val_accuracy: 0.9340 - val_loss: 0.5697 - learning_rate: 1.0000e-04
Epoch 25/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 15s 407ms/step - accuracy: 0.9388 - loss: 0.4602 - val_accuracy: 0.9375 - val_loss: 0.5495 - learning_rate: 1.0000e-04
Epoch 26/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 14s 400ms/step - accuracy: 0.9054 - loss: 0.4714 - val_accuracy: 0.9375 - val_loss: 0.5390 - learning_rate: 1.0000e-04
Epoch 27/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 15s 420ms/step - accuracy: 0.9162 - loss: 0.4506 - val_accuracy: 0.9375 - val_loss: 0.5213 - learning_rate: 1.0000e-04
Epoch 28/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 14s 400ms/step - accuracy: 0.9279 - loss: 0.4502 - val_accuracy: 0.9479 - val_loss: 0.5048 - learning_rate: 1.0000e-04
Epoch 29/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 14s 402ms/step - accuracy: 0.9116 - loss: 0.4530 - val_accuracy: 0.9444 - val_loss: 0.4979 - learning_rate: 1.0000e-04
Epoch 30/30
36/36 ━━━━━━━━━━━━━━━━━━━━ 14s 398ms/step - accuracy: 0.9216 - loss: 0.4341 - val_accuracy: 0.9479 - val_loss: 0.4770 - learning_rate: 1.0000e-04
```



```
In [ ]: # Evaluate transfer model
test_loss_tl, test_acc_tl = transfer_model.evaluate(test_ds, verbose=0)
print(f"Transfer model – Test loss: {test_loss_tl:.4f}")
print(f"Transfer model – Test accuracy: {test_acc_tl:.4f}")

y_true_tl, y_pred_tl = get_predictions_and_labels(test_ds, transfer_model)

print("\nClassification report – Transfer model:\n")
print(classification_report(y_true_tl, y_pred_tl, target_names=class_names))

cm_tl = confusion_matrix(y_true_tl, y_pred_tl)
print("Confusion matrix – Transfer model:\n", cm_tl)

plot_confusion_matrix(cm_tl, class_names, normalize=False, title="Transfer – Confusion Matrix (Counts)")
plot_confusion_matrix(cm_tl, class_names, normalize=True,  title="Transfer – Confusion Matrix (Normalized)")
```

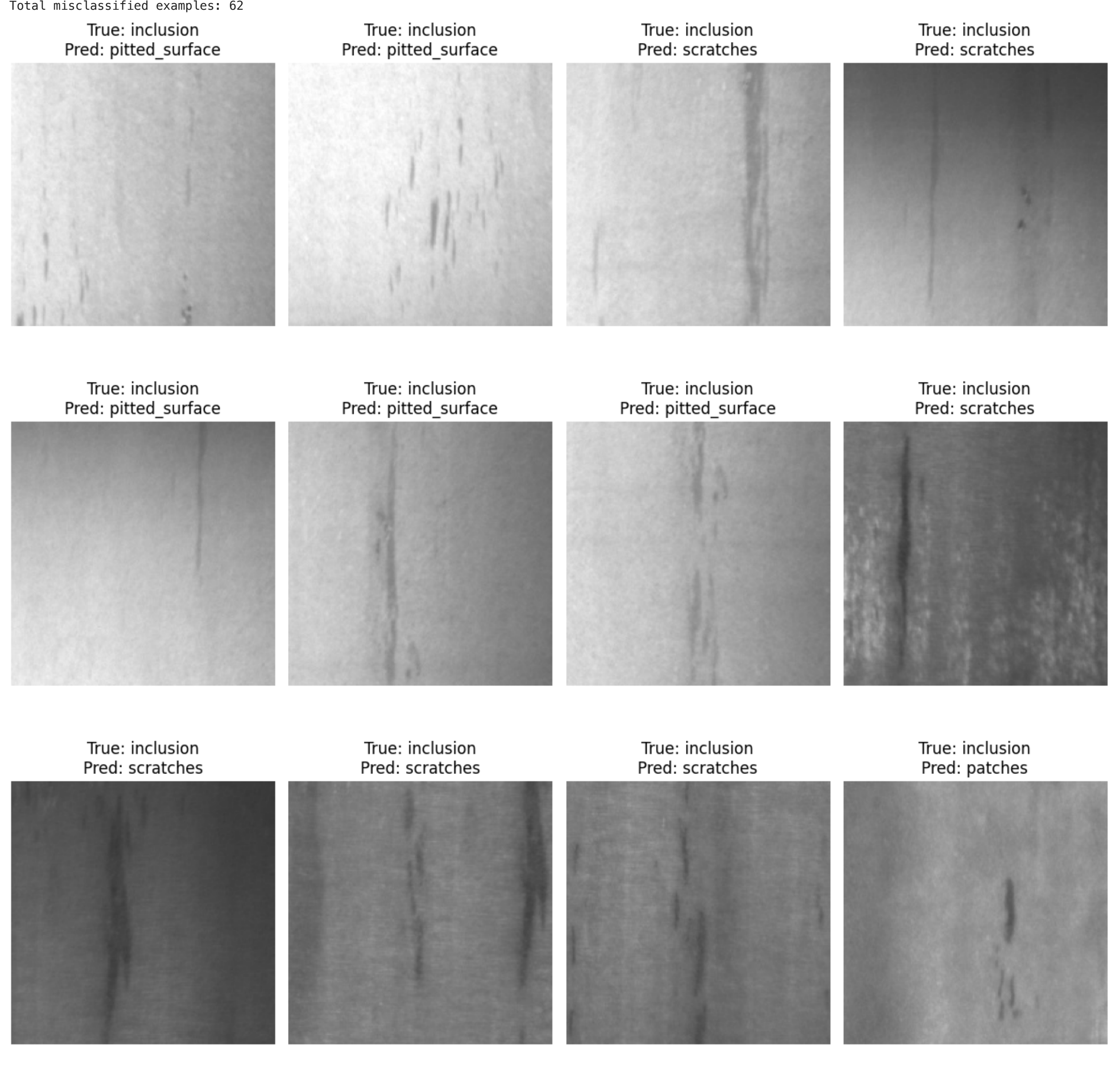
```
Transfer model – Test loss: 0.6595
Transfer model – Test accuracy: 0.8278
2025-11-09 22:07:06.364415: W tensorflow/core/framework/local_rendezvous.cc:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
Classification report – Transfer model:

                 precision    recall  f1-score   support

        crazing       0.88      1.00      0.94        60
      inclusion       0.89      0.80      0.84        60
        patches       0.95      0.98      0.97        60
 pitted_surface       0.87      0.57      0.69        60
rolled-in_scale       0.63      0.98      0.77        60
      scratches       0.86      0.63      0.73        60

       accuracy                           0.83       360
      macro avg       0.85      0.83      0.82       360
   weighted avg       0.85      0.83      0.82       360

Confusion matrix – Transfer model:
 [[60  0  0  0  0  0]
 [ 0 48  1  5  0  6]
 [ 0  0 59  0  1  0]
 [ 7  4  2 34 13  0]
 [ 1  0  0  0 59  0]
 [ 0  2  0  0 20 38]]
```





```
In [ ]: # Error analysis – visualize misclassified examples (transfer model)

# Convert test dataset to NumPy for visualization
X_test_vis, y_test_vis = X_test_np, y_test_np
y_true_vis = np.argmax(y_test_vis, axis=1)

preds_vis = transfer_model.predict(X_test_vis, verbose=0)
y_pred_vis = np.argmax(preds_vis, axis=1)

mis_idx = np.where(y_true_vis != y_pred_vis)[0]
print("Total misclassified examples:", len(mis_idx))

num_to_show = min(12, len(mis_idx))
plt.figure(figsize=(12, 12))
for i in range(num_to_show):
    idx = mis_idx[i]
    ax = plt.subplot(3, 4, i + 1)
    plt.imshow(X_test_vis[idx].astype("uint8"), cmap="gray")
    plt.title(f"True: {class_names[y_true_vis[idx]]}\nPred: {class_names[y_pred_vis[idx]]}")
    plt.axis("off")
plt.tight_layout()
plt.show()
```

```
Total misclassified examples: 62
```



Conclusion

This project applied deep learning methods to the classification of steel surface defects using the NEU-DET dataset. Starting from simple baselines (logistic regression, MLP), I demonstrated that classical models on flattened pixels perform poorly due to the complex, texture-based nature of surface defects. Custom CNNs substantially improved performance by learning spatial features, and the regularized CNN v2 introduced data augmentation and regularization techniques to better manage overfitting.